C PROGRAMMING | STUDY GUIDE

UNIONS

Concept · Syntax · Memory · Applications · Programs

- ✔ Complete theory with memory diagrams
- ✔ Comparison tables (Array vs Structure vs Union)
- ✔ Solved programs with output
- ✔ Exam tips, common mistakes & quick revision notes
- ✔ Designed for University Exams, Competitive Exams & Teacher Recruitment Tests

A concise yet comprehensive exam-oriented reference on the C "Unions" syllabus

Table of Contents

1. Concept of Union Collection 3
2. Syntax of Declaration and Its Use —
3. Comparison of Array, Structure and Union —
4. Array of Unions —
5. Union as a Member of Structure —
6. Structure as a Member of Union and Array as Member of Union —
7. Concept of Memory Saving and Union —
8. Union as a Generic Data Type —
9. Concept of Anonymous Union —
10. Programming Examples —
11. Quick Revision —

How to use this guide: Each topic is self-contained with definition, syntax, working mechanism, memory diagram, solved example with output, and exam tips — exactly following the given syllabus order. No topics outside the syllabus are included.

1. Concept of Union Collection

1.1 Definition of Union

A **union** is a user-defined derived data type in C that allows storing **different data types in the same memory location**. Unlike a structure, where each member gets its own separate memory, all members of a union **share a single common memory area**. Only one member can hold a valid value at any given point in time.

> **Key Idea:** A union is like a single "container" that can be interpreted as an `int`, a `float`, a `char` array, etc. — but only one interpretation is valid at once, because they all physically overlap the same bytes.

1.2 Characteristics of Union

- All members share the **same memory address** (offset 0).
- Size of union = size of its **largest member** (plus padding for alignment, if any).
- Writing to one member overwrites the data of all other members.
- Only **one member is active/meaningful** at a time.
- Declared using the `union` keyword, syntactically similar to a structure.
- Union variables can be assigned, passed to functions, and returned from functions, just like structures.
- Two union variables of the **same type CANNOT be compared** directly using `==` or `!=` (just like structures) — individual members must be compared instead.

> **Offset** = the distance (in bytes) of a piece of data from the *start* of a block of memory.

1.3 Memory Sharing Concept

When a union is declared, the compiler allocates a block of memory large enough to hold its **largest member**. Every member's address starts at the **same base address**. This is fundamentally different from a structure, where each member is allocated its own distinct memory, one after another.

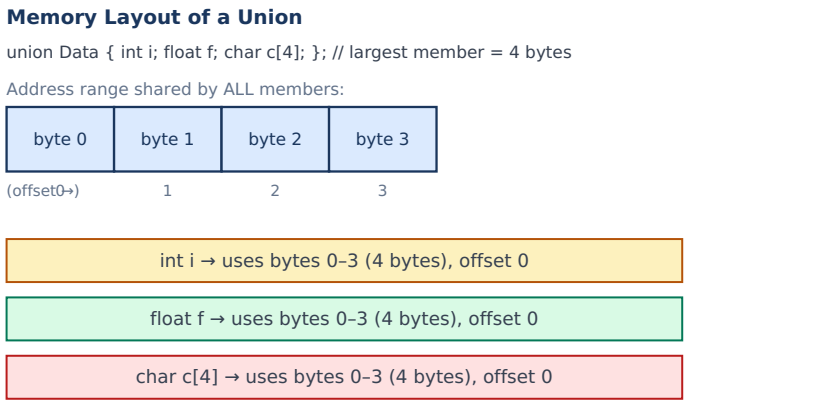


Fig 1.1 — All members (i, f, c) start at the same base address (offset 0) and overlap the same 4 bytes.

1.4 Advantages and Limitations

Advantages

- Saves memory by sharing storage among members.
- Useful for variant/generic data (data whose type isn't fixed) representation.
- Convenient for interpreting the same bits in different ways (type-punning).
- Reduces memory footprint in embedded systems & large data structures.

Limitations

- Only one member is valid at a time — data of other members is lost/corrupted.
- No built-in mechanism to know which member is currently active (must be tracked manually, e.g. with a tag).
- Cannot safely store multiple values simultaneously.
- Debugging is harder since overlapping memory can cause confusing bugs.

> **Memory footprint** = the total amount of memory (RAM) a program or data structure takes up while running.

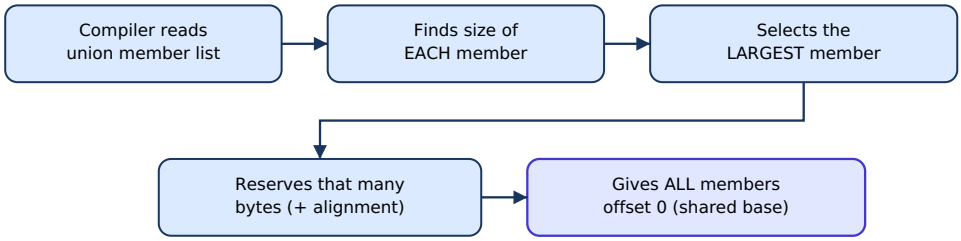
1.5 Applications of Union

- Implementing **variant records / tagged unions** (a value that can be one of several types).
- Memory-constrained embedded and system programming.
- Interpreting raw memory/bytes differently (e.g., reading a float as raw bytes for network transmission).
- Building simple **generic data type** containers.
- Hardware register mapping in device drivers.

1.6 Working Mechanism — How the Compiler Builds a Union

Whenever a union variable is declared, the compiler follows a fixed sequence of steps to allocate and manage its memory. Understanding this sequence explains *why* unions behave the way they do.

Working Mechanism — Union Creation Flowchart



Result: at runtime, writing to any member modifies the same bytes; the value last written determines what the memory currently represents — the compiler does NOT track which member was last used.

Fig 1.2 — Step-by-step working mechanism followed by the compiler when a union variable is created.

1.7 Common Mistakes

Mistake: Believing a union can store multiple members' values at the same time — it cannot; each write overwrites the previous member's bytes.

Mistake: Trying to compare two union variables directly with `==` — C does not allow whole-union/whole-structure comparison; compare member-by-member instead.

Mistake: Forgetting that the union's size depends on the **largest member**, not the sum — leads to wrong `sizeof()` predictions in exams.

Section 1 Summary

- Union = derived data type where members **share the same memory**.
- Size of union = size of the **largest member**.
- Only **one member valid** at a time.
- Saves memory but sacrifices simultaneous multi-member storage.

2. Syntax of Declaration and Its Use

2.1 Union Declaration Syntax

SYNTAX

```
union union_name {
    data_type member1;
    data_type member2;
    ...
    data_type memberN;
};
```

2.2 Creating Union Variables

SYNTAX

```
// Method 1: Declare tag first, variables later
union union_name var1, var2;

// Method 2: Declare variables along with the union
union union_name {
    int i;
    float f;
} var1, var2;

// Method 3 (typedef)
typedef union {
    int i;
    float f;
} Data;
Data var1;
```

2.3 Initializing Unions

A union can be initialized using an initializer list, but **only the first member** can be initialized this way in standard (pre-C99) style. C99 introduced **designated initializers** to initialize any specific member.

SYNTAX

```
union Data {
    int i;
    float f;
    char c;
};

union Data d1 = {10};          // initializes first member 'i' only
union Data d2 = {.f = 3.14};   // C99 designated initializer for 'f'
```

2.4 Accessing Union Members

Union members are accessed using the **dot (.) operator** for normal variables and the **arrow (->) operator** for pointers to unions — exactly like structures.

Example Program

EXAMPLE.C

```
#include <stdio.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;

    data.i = 10;
    printf("data.i : %d\n", data.i);

    data.f = 220.5;
    printf("data.f : %.1f\n", data.f);   // data.i is now corrupted
```

```
    strcpy(data.str, "C Union");
    printf("data.str : %s\n", data.str); // data.f is now corrupted

    printf("Size of union Data : %lu\n", sizeof(data));
    return 0;
}
```

OUTPUT

```
data.i : 10
data.f : 220.5
data.str : C Union
Size of union Data : 20
```

Explanation: Each assignment overwrites the same memory block. After `strcpy`, the earlier values of `i` and `f` are no longer meaningful because they shared the same 20 bytes reserved for `str` (the largest member).

Common Mistake: Reading a member right after writing a different member and expecting correct data. Always read **only the member most recently written**.

2.5 Accessing Union Members via Pointers

When a pointer to a union is available, its members are accessed using the **arrow operator (->)** instead of the dot operator. This is common when unions are passed to functions by address, or embedded inside dynamically-allocated structures.

Example Program — Pointer to Union

UNION_POINTER.C

```
#include <stdio.h>

union Data {
    int i;
    float f;
};

int main() {
    union Data d;
    union Data *ptr = &d;   // pointer to union

    ptr->i = 25;             // arrow operator used via pointer
    printf("Using pointer: ptr->i = %d\n", ptr->i);
    printf("Using variable: d.i  = %d\n", d.i);

    (*ptr).f = 5.5f;         // equivalent alternative to ptr->f
    printf("Using pointer: ptr->f = %.1f\n", ptr->f);

    return 0;
}
```

OUTPUT

```
Using pointer: ptr->i = 25
Using variable: d.i  = 25
Using pointer: ptr->f = 5.5
```

Rule: `ptr->member` is exactly equivalent to `(*ptr).member`. Both access the same shared memory as the original union variable `d`, since `ptr` merely stores `d`'s address.

2.6 Restrictions on Union Usage

- Two unions of the same type **cannot be compared** with `==` or `!=` — compare members individually instead.
- Only **one member** should be relied upon at any time; reading a "stale" member gives undefined/garbage results.
- In pre-C99 style, only the **first member** can be set via a brace initializer list.

2.7 Advantages and Limitations of Union Syntax

Advantages

- Syntax is nearly identical to structures — easy to learn and adopt.
- Supports the same access operators (`.` and `->`) as structures.
- typedef usage removes the need to repeat the `union` keyword.

Limitations

- Classic initializer lists can set only the first member (without C99 designated initializers).
- No compiler warning when reading an inactive/stale member.
- Cannot be compared directly with relational/equality operators.

Common Mistake: Writing `if (u1 == u2)` to compare two unions — this is a **compile-time error** in C; unions (like structures) have no built-in comparison operator.

Section 2 Summary

- Declared with keyword `union`; syntax mirrors `struct`.
- Accessed via `.` (variable) or `->` (pointer).
- Only first member initializable in classic C; designated initializers (C99) allow any member.
- sizeof(union) = size of largest member (+ padding).
- Unions cannot be compared using `==` / `!=`.

3. Comparison of Array, Structure and Union

3.1 Conceptual Difference

Array stores multiple elements of the **same data type** in contiguous memory. **Structure** stores multiple elements of **different data types**, each with its own separate memory. **Union** stores multiple elements of different data types but they all **share the same memory**.

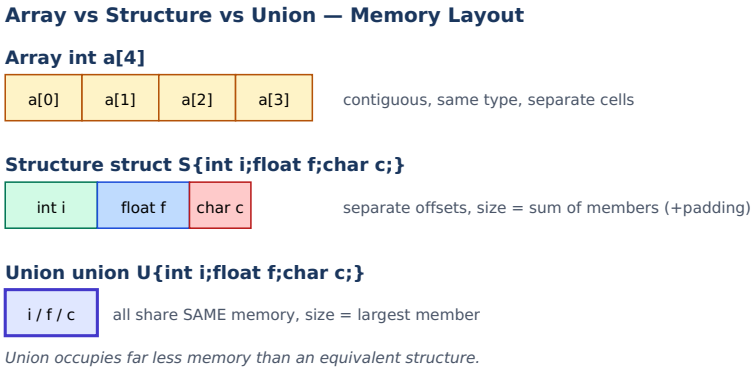


Fig 3.1 — Array = contiguous same-type cells; Structure = separate offsets; Union = overlapping shared memory.

3.2 Comprehensive Comparison Table

Aspect	Array	Structure	Union
Data type of elements	Same type only	Different types allowed	Different types allowed
Memory allocation	Contiguous, one block per element	Separate memory for each member	Single shared memory block
Total size	n × size of element type	Sum of member sizes (+padding)	Size of largest member (+padding)
Simultaneous data storage	Yes, all elements valid together	Yes, all members valid together	No, only one member valid at a time
Access method	Index: `a[i]`	Dot/Arrow: `s.mem`	Dot/Arrow: `u.mem`
Keyword	Not required (built-in)	`struct`	`union`
Memory efficiency	Efficient for homogeneous data	Less memory efficient	Most memory efficient for variant data
Typical use case	Lists, tables of same-type data	Records (e.g. student info)	Variant/tagged data, memory-saving

3.3 Advantages and Disadvantages

Type	Advantages	Disadvantages
Array	Fast indexed access, simple, cache-friendly	Fixed size, only one data type
Structure	Groups related heterogeneous data, all fields usable together	Uses more memory than union
Union	Saves memory, useful for variant types	Only one active member, risk of data overwrite

Exam Tip: A very common question is: "*What is the size of a union vs a structure with the same members?*" Remember — Structure size ≈ sum of member sizes (+padding); Union size = size of the largest member (+padding for alignment).

3.4 Decision Flowchart — Which One to Use?

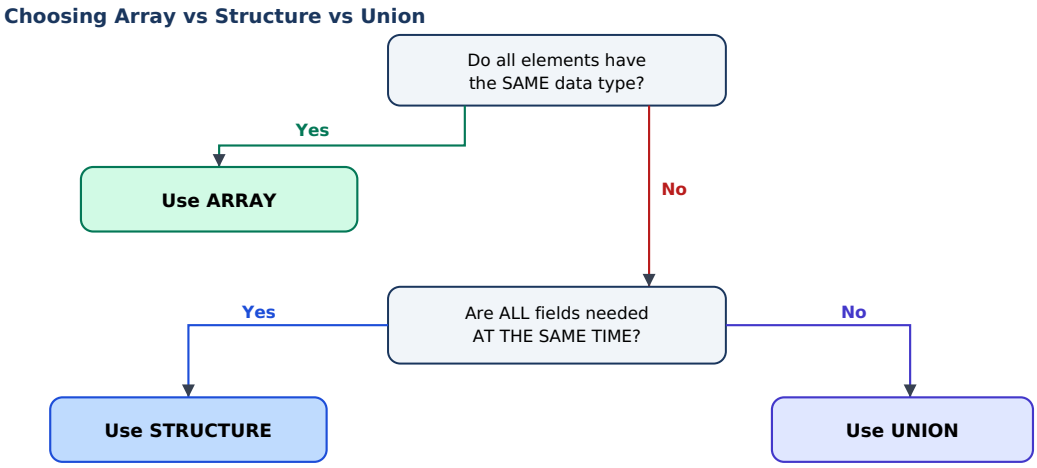


Fig 3.2 — Quick decision flowchart for choosing the right derived data type in exam problem-solving.

Common Mistake: Using a union when all fields are actually needed simultaneously (should be a structure), or using a structure when memory is critical and only one field is ever active (should be a union).

Section 3 Summary

- Array → same type, contiguous, all elements valid together.
- Structure → different types, separate memory, all members valid together.
- Union → different types, shared memory, only one member valid at a time.

4. Array of Unions

4.1 Concept

Just like arrays of structures, C allows creating an **array of union variables**. Each element of the array is an independent union — meaning each element has its own separate memory block sized to hold the largest member, but within each element the members still overlap (share memory) as usual.

4.2 Declaration, Initialization and Accessing Elements

SYNTAX

```
union Item {
    int code;
    float price;
};

union Item stock[5];              // array of 5 unions
union Item stock2[3] = { {1}, {2}, {3} };  // initializing first member of each
```

ACCESS

```
stock[0].code = 101;
stock[1].price = 99.5;
printf("%d", stock[0].code);
```

4.3 Practical Applications

- Storing a collection of variant records, e.g. mixed-type table cells in a spreadsheet.
- Implementing simple tagged-union arrays for parsers/interpreters (with a parallel "type" array).
- Memory-efficient storage of heterogeneous but mutually-exclusive data sets.

Example Program

ARRAY_OF_UNIONS.C

```
#include <stdio.h>

union Item {
    int code;
    float price;
};

int main() {
    union Item arr[3];

    arr[0].code = 101;
    arr[1].price = 49.99;
    arr[2].code = 205;

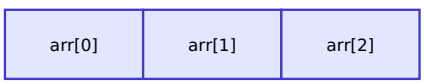
    printf("arr[0].code  = %d\n", arr[0].code);
    printf("arr[1].price = %.2f\n", arr[1].price);
    printf("arr[2].code  = %d\n", arr[2].code);

    printf("Size of arr  = %lu bytes\n", sizeof(arr));
    return 0;
}
```

OUTPUT

```
arr[0].code  = 101
arr[1].price = 49.99
arr[2].code  = 205
Size of arr  = 12 bytes
```

Memory Representation: Array of Unions (each = 4 bytes)



Each element is its own 4-byte union block; members overlap only WITHIN each element.

Fig 4.1 — Array of unions: elements are separate, but members inside each element share memory.

Note: `sizeof(arr)` = number_of_elements × size_of_one_union, since each array element still needs its own independent block.

4.4 Working Mechanism

The compiler treats `union Item stock[5]` exactly like any other array: it computes `sizeof(union Item)` once (the size of the union's largest member), then reserves **5 contiguous copies** of that block back-to-back in memory. Indexing with `stock[k]` simply moves the base pointer forward by `k × sizeof(union Item)` bytes; inside that block, the usual union member-overlap rule applies independently.

4.5 Advantages and Limitations

Advantages

- Combines array indexing convenience with per-element memory saving.
- Ideal for tables of mutually-exclusive variant values.
- Random access via index, just like a normal array.

Limitations

- Each element still needs an external tag if the active member must be tracked.
- No automatic type-safety across elements — easy to misread the wrong member.
- Total memory = n × largest member, not n × each individual member's actual size.

Common Mistake: Assuming `sizeof(arr)` equals the sum of the sizes actually used by each element's active member — it does not; every element reserves the **full union size** (largest member) regardless of which member is used.

Section 4 Summary

- Array of unions = multiple independent union blocks in sequence.
- Members overlap only within a single array element, not across elements.
- Useful for storing collections of variant/mutually-exclusive data.

5. Union as a Member of Structure

5.1 Definition

A structure can contain a union as one of its members. This is extremely useful for creating **tagged unions (variant records)** — the structure holds a "tag" field (to identify which union member is active) alongside the union itself, combining the grouping power of structures with the memory-sharing of unions.

5.2 Declaration

SYNTAX

```
union Value {
    int i;
    float f;
    char c;
};

struct Variant {
    int type;         // tag: 0 = int, 1 = float, 2 = char
    union Value data;  // union as a structure member
};
```

5.3 Accessing Union Members Inside Structures

ACCESS

```
struct Variant v;
v.type = 0;
v.data.i = 42;          // dot operator chained: struct.union_member.field
```

5.4 Applications

- Implementing **tagged unions** to safely know which member is active.
- Building interpreters / variant value types (as in scripting language engines).
- Representing hardware registers with a status tag plus data.

Example Program

UNION_IN_STRUCT.C

```
#include <stdio.h>

union Value {
    int i;
    float f;
    char c;
};

struct Variant {
    int type;            // 0=int, 1=float, 2=char
    union Value data;
};

void display(struct Variant v) {
    if (v.type == 0)
        printf("Integer: %d\n", v.data.i);
    else if (v.type == 1)
        printf("Float: %.2f\n", v.data.f);
    else
        printf("Char: %c\n", v.data.c);
}

int main() {
    struct Variant v1, v2;

    v1.type = 0;  v1.data.i = 100;
    v2.type = 1;  v2.data.f = 3.14f;

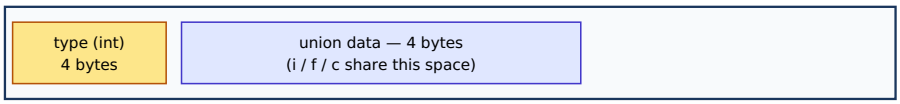
    display(v1);
    display(v2);

    printf("Size of struct Variant = %lu bytes\n", sizeof(struct Variant));
    return 0;
```

```
}
```

```
OUTPUT
Integer: 100
Float: 3.14
Size of struct Variant = 8 bytes   (4 for int type + 4 for union Value)
```

Memory Layout: Union as Member of Structure



struct Variant = [tag field][union block] — tag is separate; union members inside overlap.

Fig 5.1 — The tag field is stored separately; only the union portion shares memory internally.

Exam Tip: This "tag + union" pattern is the standard textbook way to implement a **discriminated / tagged union** in C, since plain unions have no built-in way to track which member is active.

5.5 Working Mechanism

The structure allocates its `type` field at one offset and reserves a **separate block** for the entire union at the next offset (with possible padding for alignment). The union portion internally still follows normal union rules — all its members overlap each other — but that shared block itself does **not** overlap with the structure's other (non-union) fields.

5.6 Advantages and Limitations

Advantages

- Enables safe, self-describing variant data (tag + value together).
- Groups the "what kind of data" and "the data itself" in one convenient structure.
- Foundation of tagged unions used in real compilers/interpreters.

Limitations

- Programmer must manually keep the tag in sync — the compiler does not enforce it.
- Slightly larger than a plain union due to the extra tag field.
- Incorrect tag value silently produces wrong interpretation, not a crash.

Common Mistake: Updating `v.data` without updating `v.type` (or vice versa) — this desynchronizes the tag from the actual active member, causing the wrong `if` / `switch` branch to run later.

Section 5 Summary

- A union can be embedded inside a structure as a normal member.
- Accessed by chaining: `struct_var.union_member.field` .
- Commonly paired with a "tag" field to build safe tagged unions.

6. Structure as a Member of Union and Array as Member of Union

6.1 Structure as a Member of Union — Concept

A union can also contain a **structure** as one of its members. In this case, the entire structure (with all its fields) occupies space within the union's shared memory, and it competes for space with the union's other members — the union's size becomes the size of its **largest member**, which may well be this structure.

6.2 Declaration Syntax

SYNTAX

```
struct Point {
    int x;
    int y;
};

union Shape {
    int radius;         // for circle
    struct Point p;     // structure as union member (for a point/coordinate)
};
```

6.3 Working Mechanism

When `shape.p` is written, both `p.x` and `p.y` occupy bytes inside the shared union block. Writing `shape.radius` afterwards overwrites those same bytes. The union's total size = max(size of `int radius`, size of `struct Point`).

Example Program

STRUCT_IN_UNION.C

```
#include <stdio.h>

struct Point { int x; int y; };

union Shape {
    int radius;
    struct Point p;
};

int main() {
    union Shape s;

    s.p.x = 5;
    s.p.y = 10;
    printf("Point: (%d, %d)\n", s.p.x, s.p.y);

    s.radius = 20;              // overwrites s.p entirely
    printf("Radius: %d\n", s.radius);
    printf("After overwrite, p.x = %d (garbage/undefined)\n", s.p.x);

    printf("Size of union Shape = %lu bytes\n", sizeof(s));
    return 0;
}
```

OUTPUT

```
Point: (5, 10)
Radius: 20
After overwrite, p.x = 20 (garbage/undefined)
Size of union Shape = 8 bytes
```

6.4 Array as a Member of Union — Concept

A union member may also be an **array**. This is extremely common — for example, storing a raw byte array alongside numeric types to inspect individual bytes (endianness checking, type punning).

6.5 Declaration Syntax

SYNTAX

```
union Data {
    int num;
```

```
    char bytes[4];   // array as a union member
};
```

6.6 Accessing Members

```
ACCESS
union Data d;
d.num = 1;
for (int i = 0; i < 4; i++)
    printf("%d ", d.bytes[i]);   // inspect individual bytes of num
```

6.7 Working Mechanism (Array as Union Member)

The union reserves `max(sizeof(int), sizeof(char[4]))` = 4 bytes on most systems. Since `int` and `char bytes[4]` are exactly the same size here, both fully overlap: each individual `bytes[k]` corresponds to one specific byte of the multi-byte `num`, letting the program peek at the internal byte-order (endianness) used by the machine.

Example Program — Array as Union Member (Endianness Check)

ARRAY_IN_UNION.C

```
#include <stdio.h>

union Data {
    int num;
    char bytes[4];
};

int main() {
    union Data d;
    d.num = 1;

    if (d.bytes[0] == 1)
        printf("Little Endian Machine\n");
    else
        printf("Big Endian Machine\n");

    for (int i = 0; i < 4; i++)
        printf("bytes[%d] = %d\n", i, d.bytes[i]);

    return 0;
}
```

```
OUTPUT (ON A TYPICAL LITTLE-ENDIAN PC)
Little Endian Machine
bytes[0] = 1
bytes[1] = 0
bytes[2] = 0
bytes[3] = 0
```

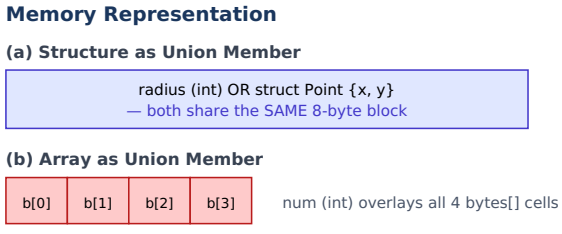


Fig 6.1 — (a) a whole structure can share union memory; (b) an array member's individual bytes overlap a numeric member.

Note: When a structure is placed inside a union, structure-padding rules still apply to the structure itself; the union then reserves space equal to that (padded) structure size if it is the largest member.

Common Mistake: Assuming a structure member inside a union keeps its own separate memory from other union members — it does not; it shares the same base address like any other union member.

6.8 Advantages and Limitations

Advantages

- Allows grouping related fields (structure) while still saving memory (union) when needed.

Limitations

- A large structure member forces the whole union to be equally large.
- Byte-level access via array members is compiler/platform dependent

- Array-in-union is the standard trick for byte-level inspection (endianness, type-punning).
- Flexible — any derived type (struct, array) can be a union member, not just primitives.

(endianness varies).
- Overwriting one member silently corrupts a nested structure/array member's data.

Section 6 Summary

- A structure can be a union member — the whole structure occupies shared union memory.
- An array can be a union member — commonly used to inspect raw bytes / type-punning / endianness checks.
- Union size = size of largest member, whether that member is a simple type, struct, or array.

7. Concept of Memory Saving and Union

7.1 Shared Memory Mechanism

The core reason unions save memory is that they **reuse the same physical bytes** for multiple logically-different members, instead of allocating separate space for each, as a structure would. This is beneficial whenever only one of several possible fields is needed at any given time.

7.2 Size Calculation

```
RULE
sizeof(union) = sizeof(largest member)  [+ padding for alignment]
sizeof(struct) = sum of sizeof(each member) [+ padding between/after members]
```

Example — Comparing Sizes

MEMORY_SAVING.C

```c
#include <stdio.h>

struct S {
    int i;      // 4 bytes
    float f;    // 4 bytes
    char c;     // 1 byte (+ padding)
};

union U {
    int i;      // 4 bytes
    float f;    // 4 bytes
    char c;     // 1 byte
};

int main() {
    printf("Size of struct S = %lu bytes\n", sizeof(struct S));
    printf("Size of union U  = %lu bytes\n", sizeof(union U));
    return 0;
}
```

```
OUTPUT (TYPICAL 32/64-BIT COMPILER)
Size of struct S = 12 bytes   (4 + 4 + 1, padded to 12)
Size of union U  = 4 bytes    (size of largest member: int/float)
```

7.3 Memory Optimization in Practice

- Use unions when a data field can logically be **"one of several types"** but never all simultaneously.
- Common in **compilers/interpreters** for token/value representation.
- Common in **embedded systems** where RAM is extremely limited.
- Used in **network packet parsing** to interpret the same buffer as different header formats.

7.4 Comparison with Structures

Criterion	Structure	Union
Memory used	High (sum of all members)	Low (only largest member)
Data availability	All members available anytime	Only last-written member valid
Best suited for	Records needing all fields together	Variant data needing only one field at a time

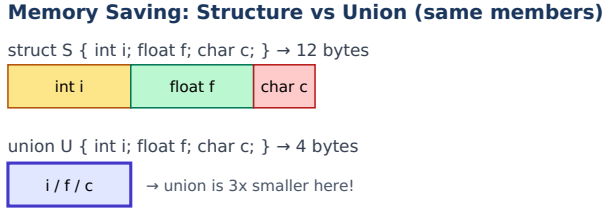


Fig 7.1 — Union reuses memory across members; structure allocates separately for each.

Exam Tip: A frequently asked numerical question: "Given a union/structure with members of sizes 4, 4, and 1 bytes, find sizeof() of each." Structure answer depends on

padding rules (usually rounds up to the largest member's alignment); union answer is simply the largest member's size (rounded to its own alignment).

Common Mistake: Assuming memory saving is "free" — the trade-off is that only one member's data survives at a time; memory saving and simultaneous multi-field access are mutually exclusive goals.

Common Mistake: Forgetting that alignment padding can make `sizeof(union)` slightly larger than the raw largest-member size (e.g., a union with a `double` member may be padded to a multiple of 8 bytes).

Section 7 Summary

- Union memory saving = reusing bytes rather than allocating new bytes per member.
- sizeof(union) = sizeof(largest member) [+ alignment padding].
- Trade-off: memory saved vs. inability to hold all members' data simultaneously.

8. Union as a Generic Data Type

8.1 Definition

Because a union can hold **any one of several data types** in the same memory space, it can act as a simple form of a **"generic" data type** — a single variable capable of representing an `int`, `float`, `char`, pointer, or other type, as chosen at runtime.

8.2 Concept of Generic Data Storage

By combining a union with a **type tag** (usually an `enum` or `int`), a program can create a variable that behaves similarly to a "variant" type found in dynamically typed languages — this is the classic C technique before generics/templates.

SYNTAX

```
typedef enum { TYPE_INT, TYPE_FLOAT, TYPE_STRING } DataType;

typedef struct {
    DataType type;
    union {
        int i;
        float f;
        char *s;
    } value;
} GenericValue;
```

8.3 Practical Applications

- Implementing **generic containers** (e.g., a generic linked list node holding any type).
- Building **interpreters and virtual machines** where a "Value" type can be int/float/string/etc.
- JSON-like or configuration-value representations in C.
- Function argument passing where the type is decided at runtime.

Example Program

GENERIC_TYPE.C

```
#include <stdio.h>

typedef enum { TYPE_INT, TYPE_FLOAT, TYPE_STRING } DataType;

typedef struct {
    DataType type;
    union {
        int i;
        float f;
        char s[20];
    } value;
} GenericValue;

void print_value(GenericValue g) {
    switch (g.type) {
        case TYPE_INT:    printf("Integer: %d\n", g.value.i); break;
        case TYPE_FLOAT:  printf("Float: %.2f\n", g.value.f); break;
        case TYPE_STRING: printf("String: %s\n", g.value.s); break;
    }
}

int main() {
    GenericValue g1 = { TYPE_INT, .value.i = 42 };
    GenericValue g2 = { TYPE_FLOAT, .value.f = 3.14f };
    GenericValue g3;
    g3.type = TYPE_STRING;
    strcpy(g3.value.s, "Hello");

    print_value(g1);
    print_value(g2);
    print_value(g3);
    return 0;
}
```

OUTPUT

```
Integer: 42
Float: 3.14
String: Hello
```

Key Idea: The union provides the "storage flexibility"; the enum tag provides "type safety" — together they emulate a generic/variant data type in C, which does not have true generics like C++ templates.

8.4 Working Mechanism

At runtime, the program first checks the `type` (enum tag) field to decide **how to interpret** the shared union bytes, then reads the matching member (e.g., `value.i` for `TYPE_INT`). This tag-then-read sequence is exactly how dynamically-typed language interpreters implement their internal "Value" objects using plain C unions.

8.5 Advantages and Limitations

Advantages

- Emulates generic/variant types without needing C++ templates or void pointers.
- Memory-efficient — one shared block regardless of how many possible types are supported.
- Widely used, well-understood pattern in real compilers and interpreters.

Limitations

- No compile-time type checking — a wrong tag leads to silently wrong behaviour, not an error.
- Every access requires manual tag-checking logic (extra boilerplate vs. real generics).
- Cannot hold two different types' data simultaneously, even in a "generic" container.

Common Mistake: Reading `g.value.f` when `g.type` is actually `TYPE_INT` — the bytes are reinterpreted as a float, producing a meaningless result instead of a compiler error.

Section 8 Summary

- Union + tag = simple generic/variant type mechanism in C.
- Widely used to implement interpreters, VMs, and generic containers.
- Type must be tracked manually — C provides no automatic type safety for unions.

9. Concept of Anonymous Union

9.1 Definition

An **anonymous union** is a union declared **without a tag name and without a variable name**. Its members become directly accessible in the enclosing scope, as if they were ordinary variables (or, inside a structure, as if they were direct members of that structure) — without needing to qualify them with a union variable name.

9.2 Syntax

SYNTAX — ANONYMOUS UNION INSIDE A STRUCTURE

```
struct Employee {
    char name[20];
    union {          // no tag, no variable name
        int id;
        char code[6];
    };               // members 'id' and 'code' accessed directly
};

struct Employee e;
e.id = 101;          // no need for e..id
```

SYNTAX — STANDALONE ANONYMOUS UNION (FILE/BLOCK SCOPE)

```
union {
    int i;
    float f;
} ;   // members i, f become directly visible in this scope (C11)
```

9.3 Features

- No tag name, no instance name for the union itself.
- Members are accessed directly using the enclosing structure's variable name (or directly in the current scope).
- Standardized in **C11** (many compilers supported it earlier as an extension).
- Reduces syntax verbosity when a union is nested inside a structure.

9.4 Applications

- Simplifying access to variant fields nested inside structures (very common in kernel/driver code).
- Hardware register definitions where bit-fields/union members must be accessed without extra qualifiers.
- Reducing boilerplate in tagged-union implementations.

> **Boilerplate** = repetitive code you have to keep re-typing that adds no real value — just friction.
> Anonymous unions cut that friction: instead of writing `e.u.id` everywhere, you just write `e.id`.

Example Program

ANONYMOUS_UNION.C

```
#include <stdio.h>

struct Employee {
    char name[20];
    int type;              // 0 = ID number, 1 = code string
    union {                 // anonymous union
        int id;
        char code[6];
    };
};

int main() {
    struct Employee e;
    strcpy(e.name, "Anita");
    e.type = 0;
    e.id = 5021;            // direct access, no union variable name needed

    printf("Name: %s\n", e.name);
    printf("ID  : %d\n", e.id);

    e.type = 1;
    strcpy(e.code, "EMP99"); // overwrites same memory as 'id'
```

```
    printf("Code: %s\n", e.code);

    return 0;
}
```

```
OUTPUT
Name: Anita
ID  : 5021
Code: EMP99
```

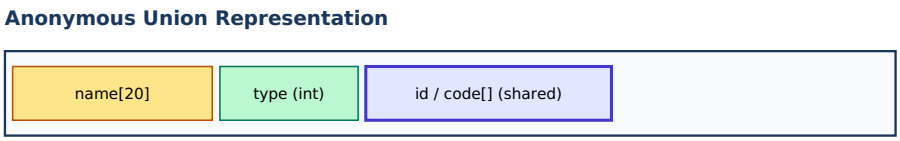


Fig 9.1 — Anonymous union members merge directly into the enclosing structure's namespace.

Exam Tip: Remember the distinction: a **named union member** requires `e.u.id` ; an **anonymous union member** is accessed simply as `e.id` .

9.5 Advantages and Limitations

Advantages

- Shorter, cleaner syntax — no intermediate union-variable name needed.
- Very natural for tagged-union patterns (tag + anonymous union together).
- Widely used in real-world kernel, driver, and hardware-register code.

Limitations

- Requires C11 (or compiler-specific extension) — not available in strict C89/C90.
- Member names must not clash with other names already in the enclosing scope.
- Slightly less explicit/readable to newcomers unfamiliar with the feature.

Common Mistake: Trying to declare a standalone (file-scope) anonymous union in a compiler that only supports the C11 feature for anonymous unions nested inside structures — support varies, so always check the compiler/standard version.

Section 9 Summary

- Anonymous union = union without tag/variable name.
- Members accessed directly in enclosing scope (standardized in C11).
- Simplifies syntax for tagged unions and register/driver-style code.

10. Programming Examples

This section consolidates representative, exam-ready programs for each concept covered in the syllabus. Each program includes explanation and expected output.

10.1 Basic Union Program

BASIC_UNION.C

```c
#include <stdio.h>

union Test {
    int x;
    char c;
};

int main() {
    union Test t;
    t.x = 65;
    printf("t.x = %d\n", t.x);
    printf("t.c = %c\n", t.c);   // shares same byte(s) as x
    printf("Size = %lu\n", sizeof(t));
    return 0;
}
```

OUTPUT

```
t.x = 65
t.c = A
Size = 4
```

Explanation: `t.c` reads the least-significant byte of `t.x` (65), which corresponds to ASCII 'A' on a little-endian system.

10.2 Array of Unions Program

ARRAY_UNION_DEMO.C

```c
#include <stdio.h>

union Val { int i; char c; };

int main() {
    union Val arr[3];
    for (int k = 0; k < 3; k++) arr[k].i = k + 65;
    for (int k = 0; k < 3; k++)
        printf("arr[%d].i=%d arr[%d].c=%c\n", k, arr[k].i, k, arr[k].c);
    return 0;
}
```

OUTPUT

```
arr[0].i=65 arr[0].c=A
arr[1].i=66 arr[1].c=B
arr[2].i=67 arr[2].c=C
```

10.3 Union Inside a Structure

UNION_INSIDE_STRUCT.C

```c
#include <stdio.h>

union Info { int id; float gpa; };

struct Student {
    char name[15];
    int mode;              // 0=id, 1=gpa
    union Info info;
};

int main() {
    struct Student s = {"Rahul", 1};
    s.info.gpa = 8.75f;
    printf("Name: %s, GPA: %.2f\n", s.name, s.info.gpa);
    return 0;
}
```

OUTPUT

```
Name: Rahul, GPA: 8.75
```

10.4 Structure Inside a Union

STRUCT_INSIDE_UNION.C

```c
#include <stdio.h>

struct Coord { int x, y; };

union Shape {
    struct Coord pt;
    int radius;
};

int main() {
    union Shape sh;
    sh.pt.x = 3; sh.pt.y = 4;
    printf("pt = (%d,%d)\n", sh.pt.x, sh.pt.y);
    sh.radius = 7;
    printf("radius = %d\n", sh.radius);
    return 0;
}
```

OUTPUT

```
pt = (3,4)
radius = 7
```

10.5 Anonymous Union Example

ANON_UNION_DEMO.C

```c
#include <stdio.h>

struct Packet {
    int type;
    union {
        int  ivalue;
        float fvalue;
    };
};

int main() {
    struct Packet p;
    p.type = 0;
    p.ivalue = 100;
    printf("ivalue = %d\n", p.ivalue);
    return 0;
}
```

OUTPUT

```
ivalue = 100
```

10.6 Pointer to Union

POINTER_UNION.C

```c
#include <stdio.h>

union Data { int i; float f; };

int main() {
    union Data d;
    union Data *ptr = &d;

    ptr->i = 77;
    printf("ptr->i = %d\n", ptr->i);
    return 0;
}
```

OUTPUT

```
ptr->i = 77
```

10.7 Restriction: Unions Cannot Be Compared Directly

UNION_COMPARE.C (COMPILE-TIME ERROR)

```
#include <stdio.h>

union Data { int i; float f; };

int main() {
    union Data d1, d2;
    d1.i = 5; d2.i = 5;

    if (d1 == d2)               // ERROR: invalid operands to binary ==
        printf("Equal\n");

    return 0;
}
```

COMPILER OUTPUT

```
error: invalid operands to binary == (have 'union Data' and 'union Data')
```

Correct approach: Compare the relevant member directly, e.g. `if (d1.i == d2.i)`, since C provides no built-in whole-union comparison.

10.8 Memory-Saving Demonstration

MEMORY_DEMO.C

```
#include <stdio.h>

struct SRec { int a; double b; char c; };
union  URec { int a; double b; char c; };

int main() {
    printf("struct size = %lu\n", sizeof(struct SRec));
    printf("union  size = %lu\n", sizeof(union URec));
    return 0;
}
```

OUTPUT (TYPICAL 64-BIT GCC)

```
struct size = 24
union  size = 8
```

Explanation: The union only reserves 8 bytes (size of `double`, its largest member), while the structure reserves separate space for each member plus alignment padding — a 3x memory saving in this example.

Section 10 Summary

- All core syllabus concepts demonstrated with compact, exam-ready programs.
- Each program highlights memory sharing/overwriting behaviour with actual output.

11. Quick Revision

11.1 Important Definitions (One-Line)

Union: A derived data type whose members share the same memory location; only one member holds valid data at a time.

Anonymous Union: A union without a tag/variable name whose members are accessed directly in the enclosing scope.

Tagged Union: A structure combining a "tag" field with a union, to track which member is currently valid.

Generic Data Type (via union): A union+tag combination used to represent a value that can be one of several types.

11.2 Syntax Summary

CHEAT SHEET

```
union Tag { type1 m1; type2 m2; };     // declaration
union Tag v;                            // variable
v.m1 = value;                           // access (dot)
ptr->m1 = value;                        // access via pointer (arrow)
union Tag v = {value};                  // init first member only
union Tag v = {.m2 = value};            // C99 designated init (any member)
union { type1 m1; type2 m2; };          // anonymous union (C11)
```

11.3 Memory Comparison Table (Quick Reference)

Type	Memory	Size Formula	All members valid?
Array	Contiguous	n × sizeof(type)	Yes
Structure	Separate per member	Σ sizeof(members) + padding	Yes
Union	Shared / overlapping	sizeof(largest member) + padding	No — only one

11.4 Frequently Asked Exam Questions

Q1. Why is the size of a union equal to the size of its largest member?
A. Because all members share one common memory block sized to fit the biggest member; no extra space is needed for the smaller ones.

Q2. Can all union members be used simultaneously?
A. No — only the member most recently written holds valid, defined data.

Q3. How do you know which union member is currently valid?
A. By maintaining a separate "tag" variable (often inside an enclosing structure) that records the type/member last written.

Q4. Differentiate structure and union in one line.
A. Structure allocates separate memory per member (all valid together); union allocates shared memory (only one valid at a time).

Q5. What is an anonymous union and where is it useful?
A. A union without a name whose members merge into the enclosing scope; useful for concise tagged-union and register-mapping code.

Q6. Can a union contain a structure or array as a member?
A. Yes — both are allowed; they occupy the shared union memory just like any simple-type member.

Q7. How are union members accessed through a pointer?
A. Using the arrow operator: `ptr->member`, equivalent to `(*ptr).member`.

Q8. Can two union variables be compared using `==`?
A. No — C does not support direct comparison of unions (or structures); compare individual members instead.

Q9. Why is a union preferred over a structure in memory-constrained systems?
A. Because a union only reserves space for its largest member, whereas a structure reserves space for all members combined.

11.5 One-Page Revision Notes

- **Union** = shared memory, size = largest member.
- **Struct** = separate memory, size = sum of members.
- **Array** = same-type, contiguous, indexed.
- Union can contain: simple types, arrays, structures.
- Structure can contain a union (common tagged-union pattern).
- Anonymous union (C11): no name, direct member access.
- Generic type: union + tag emulate variant/generic storage.
- Only **one member valid** at any time in a union — always track which one via a tag.
- Pointer access: `ptr->member` ≡ `(*ptr).member` .
- Unions (like structures) **cannot** be compared with `==` / `!=` .

11.6 Common Mistakes and Debugging Tips

Mistake 1: Reading a union member different from the one last written and expecting correct data — results in garbage/undefined values.

Mistake 2: Assuming `sizeof(union)` equals the sum of all member sizes (like a structure) — it does not; it equals the largest member's size.

Mistake 3: Forgetting to track the active member with a tag, leading to logic errors when the wrong member is interpreted.

Mistake 4: Initializing a union with multiple values in the initializer list expecting all to be set (only the first member is set in classic C89/90 initialization).

Mistake 5: Confusing anonymous union member access — writing `e.u.id` when the union is anonymous (should be simply `e.id`).

Final Exam Tip: When asked to compute `sizeof()` for a union with mixed members, always identify the **largest member first**, then apply the compiler's alignment/padding rule (usually rounded up to the alignment of that largest member).